

Engineering Cognition: Towards Persistent Understanding in AI Software Engineering

Mudit Sarda
18 July, 2026

1) Abstract

Modern AI coding agents have dramatically improved code generation, repository navigation, and autonomous software engineering. Yet despite these advances, long-running engineering tasks continue to suffer from a fundamental limitation: engineering understanding does not accumulate across development sessions. Architectural rationale, implementation decisions, debugging discoveries, and repository-specific knowledge are repeatedly reconstructed rather than reused, forcing agents to begin each session with little awareness of the engineering context established previously. Existing memory systems largely approach this limitation as a retrieval problem, preserving conversations, summaries, or embeddings for later access. We instead hypothesise that the underlying challenge is one of **Engineering Cognition**—the ability of an AI system to continuously accumulate, refine, and reuse engineering understanding over time.

To investigate this hypothesis, we developed two complementary research artefacts. The first is the **Engineering Memory System (EMS)**, an experimental platform designed to capture and reuse engineering understanding across software engineering sessions. The second is **EC-Bench**, a benchmark that evaluates whether accumulated engineering cognition measurably changes software engineering behaviour. Rather than assuming the hypothesis to be correct, EC-Bench compares identical engineering workflows performed both with and without accumulated engineering cognition while controlling all other experimental variables.

The results support the central hypothesis while simultaneously revealing important limitations in our initial design. Accumulated engineering cognition consistently improved architectural reasoning, planning, and long-horizon engineering continuity. However, the experiment also demonstrated that engineering understanding cannot simply be retrieved indiscriminately. Early context injection frequently degraded engineering reasoning, motivating a transition towards demand-driven retrieval. More importantly, the benchmark exposed several previously unrecognised research problems—including engineering understanding extraction, belief evolution, scope discipline, and task-dependent reasoning—that fundamentally changed our view of what an engineering cognition system should be.

Rather than presenting a finished system, this work documents the current state of an evolving research programme. The most significant contribution is not the measured performance improvement itself, but the experimental process through which several of our own assumptions were invalidated and replaced. We therefore argue that Engineering Cognition should be viewed as an open research problem whose progress depends on iterative experimentation, continual refinement, and a willingness to revise both the underlying hypotheses and the systems built upon them.

2) Introduction

Recent advances in large language models have transformed AI coding agents from code-completion assistants into capable software engineering systems. Contemporary agents can understand large repositories, generate substantial code changes, execute tools autonomously, and perform increasingly complex engineering workflows. As these capabilities improved, we observed an interesting shift, like software engineering itself. Code generation gradually ceased to be the dominant bottleneck. Instead, the limiting factor became the repeated reconstruction of engineering understanding.

Long-running engineering tasks naturally span multiple development sessions, repositories, debugging cycles, and implementation iterations. Human engineers accumulate architectural intuition, implementation rationale, debugging experience, and repository-specific knowledge over weeks or months of development, allowing future work to build upon previous understanding. In contrast, contemporary coding agents largely reconstruct this understanding from scratch whenever a new session begins. Although previous conversations may remain accessible, the accumulated engineering reasoning behind those conversations rarely persists in a form that can meaningfully influence future decisions.

Most existing approaches treat this limitation as a memory problem. Conversation persistence, retrieval systems, embeddings, repository indexing, and compressed summaries attempt to preserve previously generated information so that it can be retrieved later when required. While these techniques substantially improve context availability, they primarily preserve information rather than engineering understanding itself. The central question motivating this work is therefore not *how an agent should remember previous conversations*, but *whether engineering understanding itself can become an explicit computational object that accumulates, evolves, and improves future engineering behaviour*.

We refer to this hypothesis as **Engineering Cognition**. Rather than viewing engineering memory as a collection of stored documents, Engineering Cognition proposes that software engineering produces an evolving body of reusable understanding that should be continuously constructed, maintained, and refined independently of any individual coding session. The theoretical foundations of this hypothesis are presented in the accompanying *Engineering Cognition* paper. The objective of the present work is instead empirical: to determine whether accumulated engineering cognition produces measurable differences in software engineering behaviour.

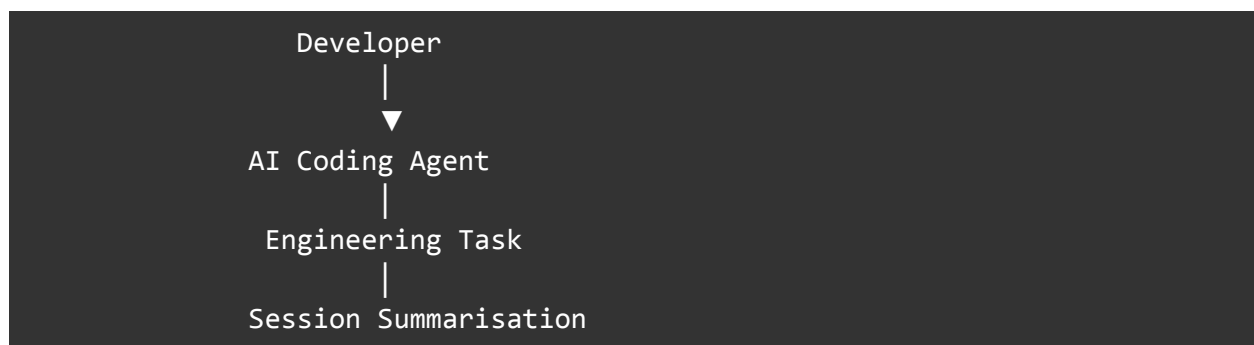
Evaluating this hypothesis required more than implementing a memory system. Without systematic experimentation, apparent improvements could easily result from prompt engineering, confirmation bias, or implementation-specific behaviour. We therefore designed the experiment before attempting to validate the implementation itself. This led to the development of **EC-Bench**, a benchmark specifically designed to compare identical long-horizon software engineering tasks performed both with and without accumulated engineering cognition while varying only a single experimental variable: the availability of previously accumulated engineering understanding.

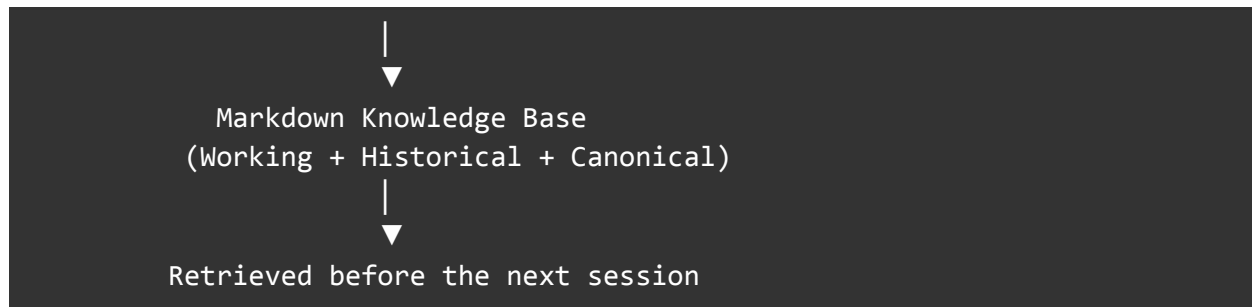
The experiment was intentionally designed to challenge our assumptions rather than confirm them. Throughout its execution, several of our original design decisions were shown to be incomplete or incorrect, leading to substantial revisions of both EMS and our broader understanding of Engineering Cognition. Consequently, the contribution of this work is not simply a benchmark or an implementation, but an experimental investigation into whether accumulated engineering understanding represents a distinct and measurable capability for future AI software engineering systems.

3) From Engineering Memory to Engineering Cognition

The original motivation behind this work was not Engineering Cognition but a much narrower problem: AI coding agents repeatedly lost engineering context between sessions. Every new session required the agent to reconstruct repository structure, architectural decisions, implementation rationale, debugging discoveries and engineering assumptions that had already been established during previous work. We initially believed that this limitation arose primarily from the absence of persistent engineering memory.

Based on this assumption, we designed the first version of the **Engineering Memory System (EMS)** as a persistent knowledge layer that continuously accumulated engineering context across software development sessions. At the end of each coding session, the agent summarised its work into structured markdown documents, proposed additions to a canonical knowledge base for developer review, and used the accumulated documentation to initialise future engineering sessions.





This architecture reflected three fundamental assumptions. First, engineering productivity would improve if important repository knowledge persisted instead of being rediscovered repeatedly. Second, engineering understanding could be represented as structured documentation organised hierarchically across repositories, branches and sessions. Finally, providing this accumulated knowledge before the agent began reasoning would enable subsequent engineering tasks to start from a more informed state.

As EMS evolved, however, the problem itself began to appear different. We gradually realised that engineering work depends not merely on remembering previous information, but on accumulating **engineering understanding**—the evolving collection of architectural intuition, implementation rationale, debugging experience and engineering beliefs that human engineers naturally develop over long periods of work. This observation motivated a broader hypothesis: perhaps software engineering requires a persistent form of **Engineering Cognition**, rather than simply persistent engineering memory.

The first EC-Bench experiment was therefore designed to evaluate two related questions. First, does accumulated engineering understanding measurably improve long-horizon software engineering? Second, were the underlying assumptions of the original EMS architecture—including document-centric storage, eager retrieval and conversation summarisation—actually sufficient to construct such understanding? The answers to these questions ultimately reshaped both our conceptual model of Engineering Cognition and the architecture of EMS itself.

4) Experimental Design and Results

4.1) Experimental Design

Engineering Cognition was evaluated using **EC-Bench**, a benchmark designed to measure whether accumulated engineering understanding changes the behaviour of an AI software engineer during a realistic long-horizon software engineering task.

Unlike conventional software engineering benchmarks, EC-Bench evaluates the complete engineering lifecycle rather than isolated code generation. The benchmark follows a fixed

sequence of thirty prompts spanning repository familiarisation, architectural reasoning, implementation, debugging, validation and future planning. Every benchmark run executes the identical prompt sequence under two experimental conditions:

- **Stateless Baseline:** repository only
- **EMS-assisted:** repository augmented with accumulated Engineering Cognition

The experiment was designed as a controlled comparison. Every variable—including the repository, model, execution environment, prompt sequence and evaluation procedure—remained constant across both conditions. The only independent variable was the availability of accumulated engineering understanding.

To evaluate engineering behaviour rather than code correctness alone, responses were scored using five engineering metrics:

- Architectural Continuity
- Repository Groundedness
- Engineering Cognition Reuse
- Engineering Quality
- Debugging & Investigation Efficiency

Each response was independently evaluated by an LLM-based Judge Agent operating under a repository-first evaluation philosophy. The judge remained blind to whether a response originated from the EMS-assisted or stateless condition, ensuring that evaluations reflected engineering behaviour rather than experimental expectations.

4.2) Experimental Results

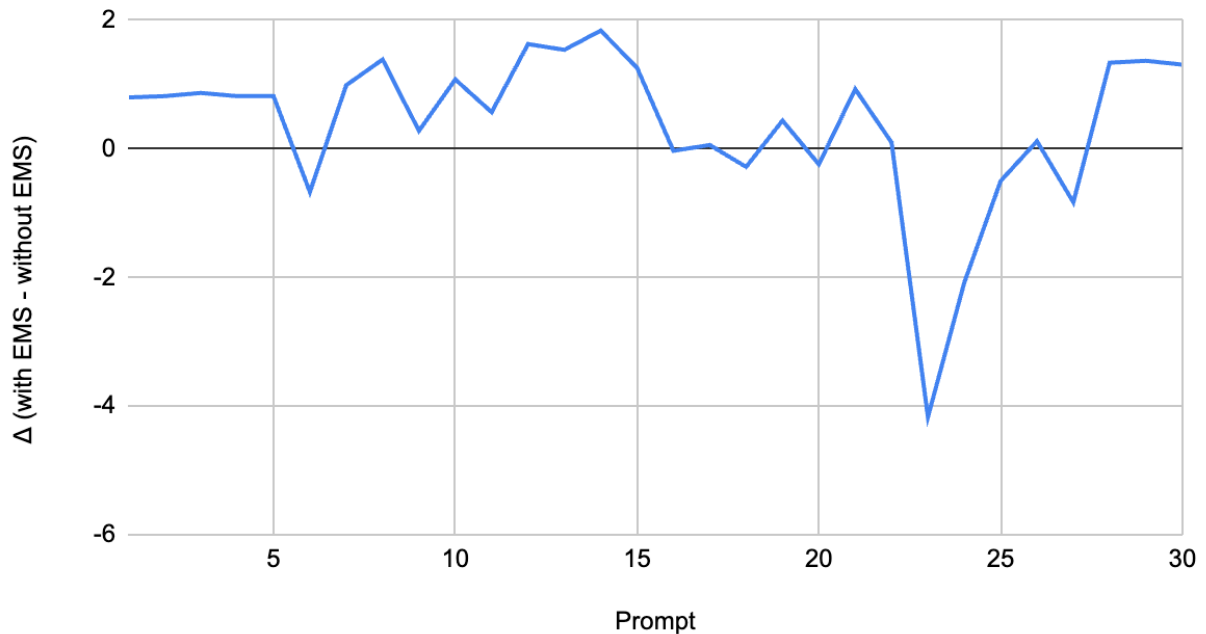
Across the complete benchmark, EMS outperformed the stateless baseline on **22 of the 30 engineering tasks**, increasing the overall weighted score from **7.96** to **8.35**, corresponding to an average improvement of **+0.37 points per prompt**.

Overall benchmark summary

Statistic	Value
Total prompts	30
EMS wins	22
Stateless wins	8
Mean score (with EMS)	8.35
Mean score (without EMS)	7.96

Mean improvement	+0.37
------------------	-------

Δ (with EMS - without EMS) vs. Prompt



Performance improvements were concentrated primarily within **repository investigation**, **architectural reasoning** and **engineering planning**, where accumulated engineering understanding consistently reduced redundant reasoning and enabled the agent to continue previously established engineering decisions.

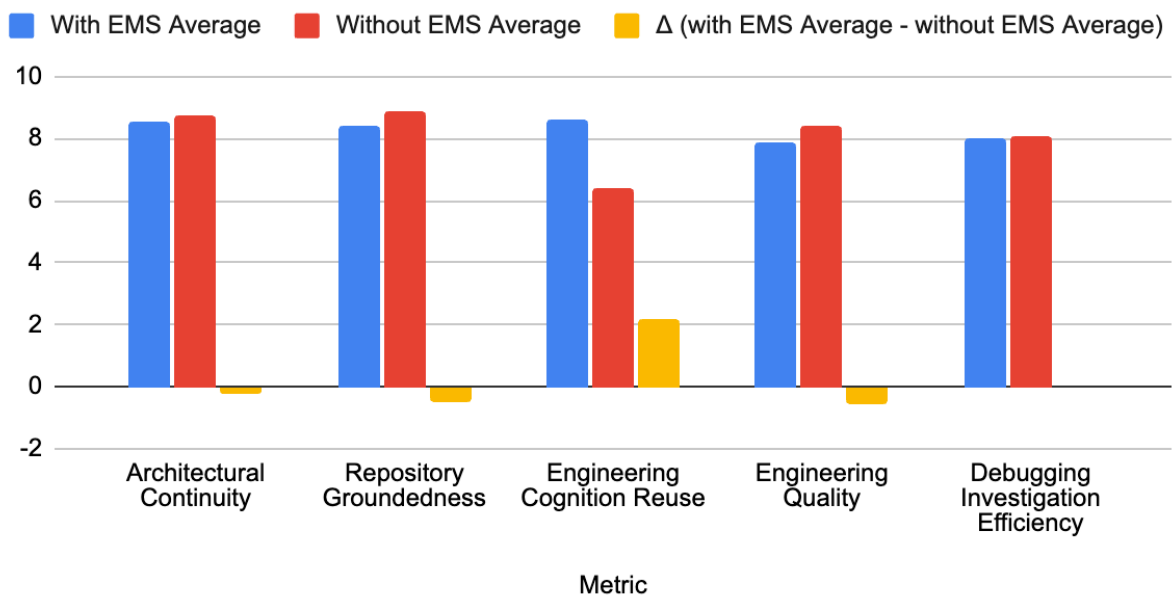
The largest improvements occurred during architectural reasoning tasks, with Prompt 14 (+1.84), Prompt 12 (+1.63) and Prompt 13 (+1.54) demonstrating that previously accumulated understanding substantially improved long-horizon engineering decision making.

Conversely, the largest regressions appeared almost exclusively during debugging tasks, particularly Prompt 23 (-4.17) and Prompt 24 (-2.08). In these cases, previously accumulated engineering knowledge occasionally influenced investigations before sufficient repository evidence had been established, causing the agent to prematurely anchor its reasoning around earlier engineering beliefs.

Metric-level analysis further revealed an unexpected result.

Metric	With EMS Average	Without EMS Average	Δ (with EMS Average - without EMS Average)
Architectural Continuity	8.543	8.766	-0.223
Repository Groundedness	8.396	8.876	-0.480
Engineering Cognition Reuse	8.590	6.376	+2.214
Engineering Quality	7.844	8.433	-0.589
Debugging Investigation Efficiency	7.993	8.050	-0.057

With EMS Average, Without EMS Average and Δ (with EMS Average - without EMS Average)



While EMS dramatically increased **Engineering Cognition Reuse**, improvements did not consistently translate into higher engineering quality across every task. This observation became the most significant outcome of the experiment and directly motivated the redesign of EMS presented in the following section.

5) What We Learned

The primary contribution of the first EC-Bench experiment extends beyond the benchmark scores themselves. While the quantitative results demonstrate that accumulated engineering cognition changes software engineering behaviour, the qualitative analysis explains *how* that behaviour changes and why certain assumptions behind the original EMS architecture proved to be incomplete.

Across the thirty-prompt workflow, four recurring patterns consistently emerged. Together, these findings reshaped both our understanding of Engineering Cognition and the subsequent redesign of EMS.

5.1) Retrieval Behaviour

Perhaps the strongest finding concerns **when** engineering understanding should influence reasoning. Our original assumption was that accumulated engineering knowledge should be retrieved before an engineering task begins, allowing the agent to reason from a richer initial context. The experiment repeatedly contradicted this assumption.

The highest-quality investigations consistently established an understanding of the current repository before consulting accumulated engineering knowledge. When retrieval occurred too early, the agent frequently anchored its reasoning on previous engineering beliefs rather than on newly observed repository evidence. This behaviour was particularly visible during debugging tasks, where premature retrieval occasionally diverted investigations toward previously established hypotheses instead of the actual implementation.

The experiment therefore suggests that retrieval timing is as important as retrieval quality. Engineering cognition should support reasoning once uncertainty emerges rather than initialise it before repository understanding has been established.

5.2) Scope Expansion

Accumulated engineering understanding also changed *what* the agent attempted to solve. As engineering cognition grew throughout the benchmark, EMS increasingly proposed repository improvements beyond the immediate task, extending implementation work into broader architectural recommendations and future optimisation opportunities.

This behaviour proved beneficial during planning and architectural reasoning, where accumulated understanding naturally enabled longer-term engineering thinking. During debugging and verification, however, the same tendency occasionally caused the agent to solve adjacent engineering problems instead of completing the requested objective.

These observations suggest that scope expansion should become an explicit engineering decision rather than an automatic consequence of accumulated cognition.

5.3) Engineering Modes

A third finding is that software engineering cannot be treated as a single cognitive activity. Investigation, architectural reasoning, implementation, debugging and planning consistently required different patterns of engineering understanding, and no single retrieval strategy performed well across every phase of development.

This indicates that future engineering cognition systems should become **mode-aware**, adapting how engineering understanding is retrieved, accumulated and applied according to the type of engineering task being performed rather than relying on a uniform memory pipeline.

5.4) What Actually Improved?

Finally, the experiment demonstrated that Engineering Cognition does not improve every aspect of software engineering equally. The largest gains consistently appeared during architectural reasoning and long-term planning, where accumulated understanding enabled the agent to continue previous engineering work instead of repeatedly reconstructing it.

Conversely, investigation and debugging depended primarily on disciplined repository verification and evidence-driven reasoning. In these situations, accumulated engineering knowledge sometimes became a source of bias rather than assistance when introduced prematurely.

Collectively, these findings suggest that the value of Engineering Cognition lies not in replacing repository analysis, but in extending engineering understanding once the current implementation has been correctly established. This distinction ultimately became the central insight that motivated the redesign of EMS presented in the following section.

6) Rethinking EMS

The first EC-Bench experiment demonstrated that the original Engineering Memory System solved a much narrower problem than we initially believed. Our original architecture treated Engineering Cognition primarily as a persistence problem: engineering knowledge produced during one coding session could be summarised, stored as structured markdown, and retrieved before the next session. The underlying assumption was that preserving engineering information would naturally allow engineering understanding to accumulate over time.

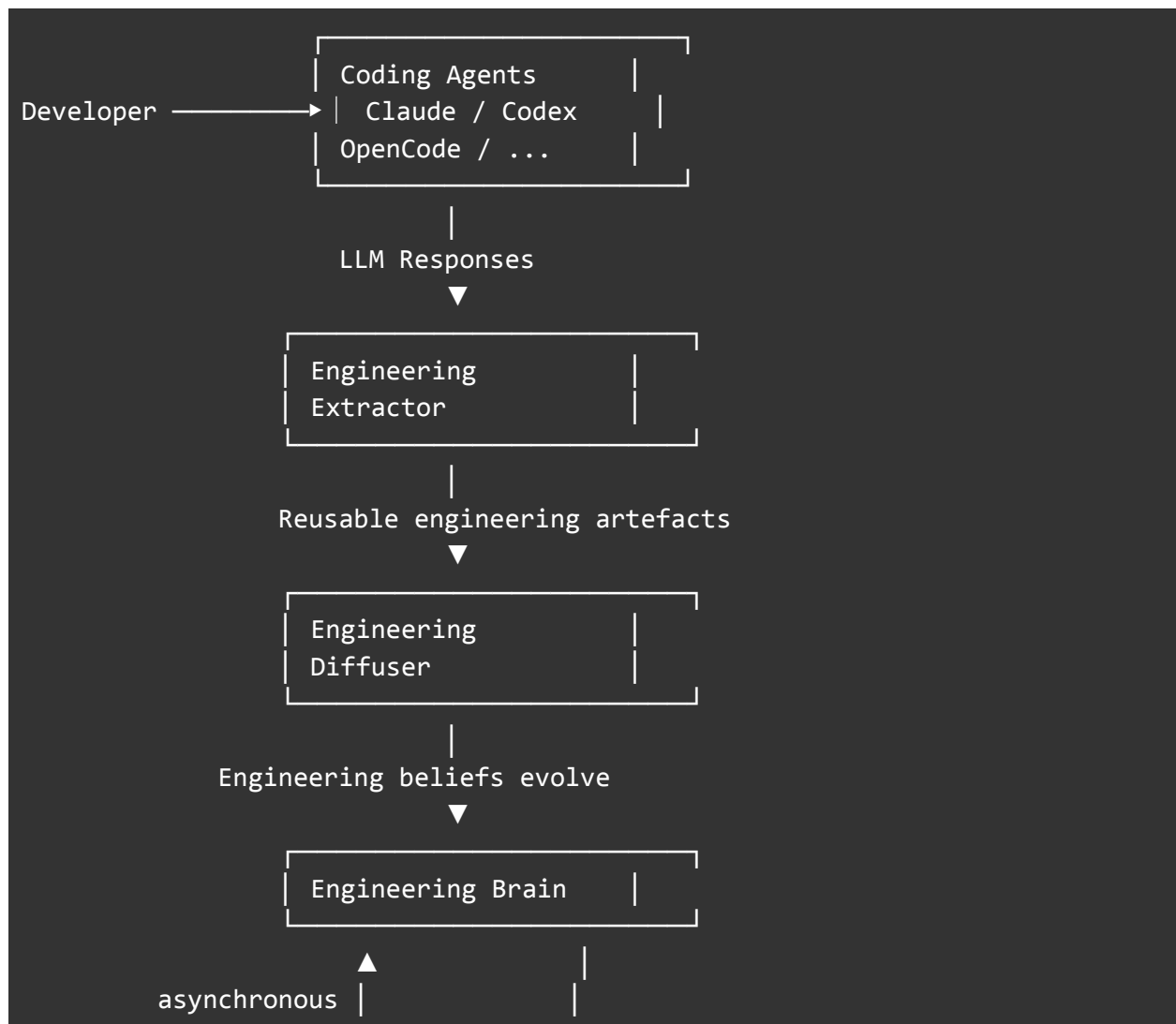
While this design successfully preserved engineering context across sessions, the experiment revealed three fundamental shortcomings. First, the system treated engineering understanding as documents rather than as evolving engineering concepts. Second, it assumed that retrieval

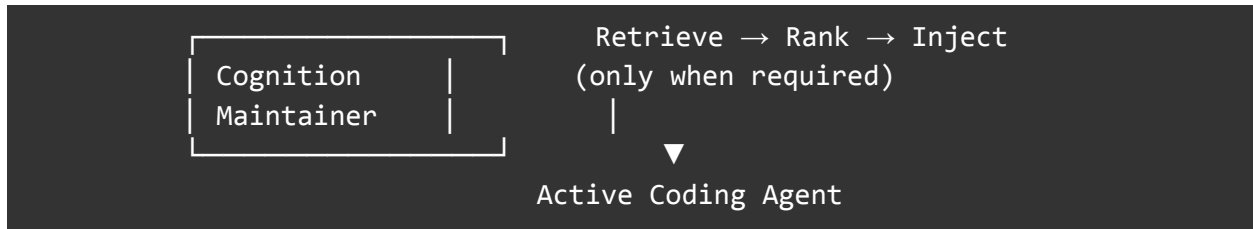
should occur before reasoning began, an assumption repeatedly contradicted by the debugging and investigation prompts. Finally, it implicitly treated engineering understanding extraction as a solved problem, assuming that summarising an LLM conversation was sufficient to preserve reusable engineering knowledge. The benchmark demonstrated that none of these assumptions consistently held under long-horizon software engineering.

These observations fundamentally changed our understanding of EMS. Rather than viewing persistent memory as the objective, we began viewing it as one stage within a broader Engineering Cognition pipeline whose purpose is to continuously construct, refine and reuse engineering understanding.

6.1) Engineering Cognition Pipeline

The revised architecture separates Engineering Cognition into five independent stages, each addressing a distinct research problem rather than a storage problem.





The **Engineering Extractor** attempts to recover reusable engineering understanding from the reasoning process of coding agents instead of simply summarising conversations. These artefacts are then consumed by the **Engineering Diffuser**, which determines whether new observations strengthen, refine, supersede or create engineering beliefs. The resulting Engineering Brain therefore stores evolving engineering concepts rather than independent markdown documents.

Because engineering understanding changes continuously, a dedicated **Cognition Maintainer** operates asynchronously, maintaining relationships between artefacts, updating confidence estimates and evolving engineering beliefs without interrupting the developer's workflow. Finally, accumulated understanding is made available through **demand-driven retrieval**, allowing repository reasoning to precede memory retrieval instead of the reverse.

6.2) Implications

The experiment, therefore, fundamentally changed our view of EMS. Initially, we believed that persistent engineering memory was the primary problem to solve. Following the benchmark, we now believe that persistent memory is only an intermediate representation within a larger Engineering Cognition pipeline.

The central challenge is no longer storing engineering information, but extracting reusable engineering understanding, allowing that understanding to evolve as evidence accumulates, and introducing it into reasoning only when it genuinely improves engineering decisions. EMS should therefore no longer be viewed as a memory system. It is more accurately understood as an experimental platform for constructing and evaluating persistent Engineering Cognition.

7) Discussion

7.1) Limitations

The results presented in this work should be interpreted within the scope of the current experimental design rather than as definitive evidence for Engineering Cognition.

First, the experiment was conducted on a single production repository and therefore reflects one engineering domain, one software architecture and one development workflow. While the

repository contains realistic investigation, architectural reasoning, implementation, debugging and planning tasks, additional studies across different languages, frameworks and repository sizes are required before broader conclusions can be drawn.

Second, both experimental conditions were evaluated using the same coding model and assessed by the same LLM-based judge. This isolates the effect of accumulated engineering cognition while minimising model variability, but it does not establish that the observed behaviours generalise across different model families or evaluation methodologies.

Finally, the Engineering Memory System evaluated here should be viewed as an experimental research prototype rather than a finished architecture. One of the principal outcomes of the benchmark was the redesign of EMS itself. Consequently, this experiment evaluates an initial implementation of Engineering Cognition, not its eventual upper bound.

7.2) Open Research Questions

Perhaps the most significant outcome of this work is not the benchmark scores themselves, but the number of new research questions exposed by the experiment. The redesign of EMS transforms Engineering Cognition from an engineering project into a broader research programme whose central challenges remain largely unexplored.

Several questions appear particularly fundamental. How can reusable engineering understanding be extracted reliably from the reasoning process of large language models rather than from their final responses? Once extracted, how should individual engineering artefacts be diffused into higher-level engineering beliefs without introducing incorrect generalisations or discarding repository-specific context? As software evolves, how should these beliefs themselves evolve, merge, or be superseded while preserving historical reasoning?

The experiment also raises questions about how Engineering Cognition should be represented and utilised. What ontology best captures engineering understanding? When should accumulated cognition influence reasoning, and when should repository evidence take precedence? Finally, should Engineering Cognition remain model-dependent, or should it be constructed by specialised extraction models capable of producing a persistent engineering understanding that can be shared across future coding agents?

Answering these questions will require substantially more experimentation than presented here. Rather than concluding the investigation, the first EC-Bench experiment establishes a foundation upon which future iterations of both EMS and Engineering Cognition can be systematically developed and evaluated.

8) Conclusion - What We Believe Today

When this work began, we believed the primary obstacle to long-running AI software engineering was **memory persistence**. If engineering knowledge could simply survive across

sessions, we expected future engineering performance to improve naturally. EMS was therefore conceived as a structured memory system that stored engineering knowledge in hierarchical markdown documents and reintroduced that knowledge at the beginning of subsequent sessions.

The first EC-Bench experiment substantially changed this perspective.

Across investigation, architectural reasoning, implementation, debugging, and planning tasks, improvements rarely resulted from *remembering more*. Instead, they emerged when previously accumulated understanding was **constructed, reused, revised, and retrieved at the appropriate moment**. Likewise, the largest regressions were not caused by missing information, but by engineering knowledge being retrieved too early or applied beyond the scope of the current task.

These observations suggest that the central challenge is not persistent memory, but **persistent engineering cognition**.

Before the experiment	What we believe today
The problem is persistent memory.	The problem is persistent engineering cognition.
Better retrieval produces better engineering.	Better extraction and belief formation produce better engineering.
Markdown documents are the primary knowledge representation.	Structured engineering artefacts and evolving engineering beliefs should become the canonical representation.
Knowledge should be retrieved at the beginning of a session.	Knowledge should be retrieved only when reasoning requires it.
Memory accumulation is the objective.	Understanding accumulation is the objective.
EMS is primarily a memory management system.	EMS should become a continuously evolving engineering cognition pipeline.
Repository context is sufficient.	Reasoning discipline determines whether accumulated understanding improves future work.

Perhaps the most important outcome of this work is that it reframes the research problem itself. We no longer view EMS as the final objective of this research. Instead, it has become an experimental platform for investigating a broader question:

Can engineering understanding itself become a computational object that accumulates, evolves, and improves software engineering over time, independently of any individual coding agent?

This report does not answer that question. It demonstrates that the question is measurable, experimentally testable, and worthy of continued investigation. The central challenge is no longer remembering engineering work—it is **constructing engineering understanding**.